ZKinsta

Instagram clone built with microservices

Project type: Full-stack microservices web application
Backend: Java 21, Spring Boot, Spring Cloud
Frontend: React 18, TypeScript
Database: MySQL 8.0 (4 separate databases)
Infra: Docker, Docker Compose, Consul

Project Summary Document

A concise overview of what's built, how the frontend works, and how the backend services talk to each other.

1. What is ZKinsta?

ZKinsta is a social-media web application — basically an Instagram clone — built to demonstrate how modern cloud-native applications are designed using a microservices architecture. Instead of one big application doing everything, the project is split into five small independent services. Each service handles one specific job, has its own database, and talks to other services over HTTP.

The frontend is a React + TypeScript single-page application. The backend is five Spring Boot services running in Docker containers, with MySQL for data and Consul for service discovery. Everything is orchestrated using Docker Compose so it runs with a single command.

Core features users get

Area	What works
Auth	Sign up, log in, password reset, view + edit profile, search users
Posts	Create posts with image/video, captions, hashtags, 12 Instagram-style filters, privacy controls (public / friends-only / private)
Engagement	Like / unlike, comment, view tracking, double-tap to like
Social	Follow / unfollow, followers + following lists, in-app notifications
Discovery	Public feed, trending hashtags, hashtag search, user search with autocomplete
UI	Dark theme, fully responsive (mobile + tablet + desktop), real-time-ish notification polling

2. Frontend — what's there and how it works

The frontend is a React 18 + TypeScript single-page application running on port 3000. It uses React Router for client-side navigation, Axios for talking to the API Gateway, and React Context for managing logged-in user state. Every screen is built from reusable components.

Tech stack at a glance

Tool	Role
React 18.3	Component-based UI, virtual DOM for fast updates, hooks for state and side-effects
TypeScript 4.9	Static typing — catches bugs at compile time, better IDE autocomplete, interfaces for every data type
React Router 6	Maps URLs to components, dynamic routes like /profile/:username, hooks like useNavigate and useParams
Axios 1.7	HTTP client with a centralised baseURL, request + response interceptors for JWT handling
Pure CSS	All filters, animations, and responsive layouts written in CSS — no UI library, no Tailwind

How the frontend is organised

The codebase is split into three layers: **components** (UI pieces like PostCard, Navbar, Profile), **services** (TypeScript modules that wrap API calls — authService, postService, followService, etc.), and **context** (global state like the logged-in user). Each component just calls the right service function and renders the result — no HTTP code mixed with UI code.

Key things that make the UI feel fast

a) Optimistic UI updates

When you like a post, the heart turns red and the count goes up **immediately**, before the server even responds. If the server call later fails, the UI rolls back. This is called optimistic updating — the user perceives the app as instant instead of waiting 200–500ms for every click.

b) Axios interceptors

Every API call goes through one central Axios instance. A request interceptor automatically attaches the JWT token from localStorage to every outgoing request, so no component has to manually handle auth headers. A response interceptor catches 401 errors (expired token) and redirects the user to login. This keeps every component clean.

c) React Context for auth state

The AuthContext stores the current user, the token, and helper functions (login, logout, refreshProfile). Any component anywhere in the tree can read this state using the useAuth() hook. ProtectedRoute uses it to redirect unauthenticated users away from private pages.

d) Image filters using just CSS and Canvas

The 12 Instagram-style filters (Clarendon, Moon, Lark, etc.) are pure CSS filter values — brightness, contrast, saturation, sepia, grayscale, hue-rotate. For live preview, the values are applied as a CSS filter property on the image. When the user clicks Apply, the same string is applied to an HTML5 Canvas context, the image is redrawn, and exported as a JPEG. No external libraries, no npm packages — works in every modern browser.

e) IntersectionObserver for view tracking

Each post card watches its own visibility using the browser's IntersectionObserver API. When 50% of the post enters the viewport, it fires a single API call to record a view. A React ref prevents duplicate calls within the same mount, and the backend also has a 5-minute cooldown per user per post.

f) Debounced search and notification polling

The search bar waits 300ms after the last keystroke before calling the API, reducing 20 requests to 1 or 2. The navbar polls the notifications endpoint every 30 seconds to update the unread count badge — a simple alternative to WebSockets that works fine at this scale.

Responsive design — three breakpoints

Breakpoint	Layout behaviour
Mobile (< 768px)	Sidebar hidden completely. Bottom fixed nav with 5 icons (Home, Search, Create, Explore, Profile). Main content full-width.
Tablet (768–1263px)	Sidebar collapses to 72px with icons only. Search and notifications slide out as side panels.
Desktop (≥ 1264px)	Full 245px sidebar with icon + text labels. Comfortable reading width for the feed.

Dark theme

The app is dark-only — no theme toggle. Achieved by setting color-scheme: dark on the html element so browsers render form controls in dark mode, then explicit dark backgrounds and light text on every container. Borders use dark grey (#262626) instead of light grey to avoid washed-out lines.

> **Frontend in one sentence:** a React + TypeScript SPA that talks to one API Gateway URL, manages auth via Context + localStorage, applies optimistic updates and debouncing for instant feel, and renders Instagram-style UI built entirely from custom components with no UI library.

3. Backend — what runs and how it's split

The backend is five independent Spring Boot 3.2 services written in Java 21. Each service owns its domain, has its own MySQL database, and exposes HTTP endpoints. None of them share tables — when one service needs data from another, it makes an HTTP call.

The five services

Service	Port	Responsibility
API Gateway	8080	Single entry point for all client requests. Routes URLs to the right service, validates JWT tokens, handles CORS, deduplicates response headers.
Authentication Service	8081	User accounts, registration, login, password reset, profile management, user search. Owns the instagram_auth database.
Post Service	8082	Posts, comments, likes, view tracking, media uploads, public feed. Owns the instagram_posts database. Stores image and video bytes as MEDIUMBLOB.
Follow Service	8083	Follow / unfollow, followers + following lists, notifications (LIKE, COMMENT, FOLLOW). Owns the instagram_follows database.
Trending Service	8084	Hashtag tracking, trending hashtag list, hashtag search. Owns the instagram_trending database.

Supporting infrastructure

Component	Role
MySQL 8.0	Stores all persistent data. Four databases, one per business service. InnoDB engine, utf8mb4 charset for emoji support.
Consul	Service registry. Every service registers on startup. Performs HTTP health checks every 10s. Gateway uses Consul to discover service IPs at request time.
Docker + Docker Compose	Each service has its own Dockerfile. Compose orchestrates startup order, networking, and the persistent MySQL volume.

4. How the backend works — services talking to each other

Step 1 — Service discovery via Consul

When any service starts, it registers itself with Consul: *here's my name, my IP, my port, and the URL where you can check if I'm healthy*. Every 10 seconds, Consul calls each service's /actuator/health endpoint. If a service is down, Consul marks it red. The API Gateway always asks Consul for the current healthy instances before forwarding requests — so if a service moves, restarts, or scales to multiple copies, the gateway adapts automatically.

Step 2 — API Gateway is the front door

The frontend never talks to backend services directly. It only knows about port 8080 — the API Gateway. The gateway has a route table that maps URL prefixes to services:

URL pattern	Forwarded to
/api/auth/**	authentication-service
/api/posts/**	post-service
/api/follows/**	follow-service
/api/notifications/**	follow-service
/api/trending/**	trending-service
/api/media/**	post-service

Step 3 — JWT authentication at the gateway

Protected routes go through the JwtAuthenticationFilter. When a request arrives with Authorization: Bearer eyJ..., the gateway validates the JWT signature, checks expiry, and extracts the username + userId from the token claims. It then forwards the request to the downstream service with two extra headers: X-Username and X-User-Id. The backend services trust these headers and never deal with JWT directly. This means only the gateway needs the JWT secret key.

Step 4 — Inter-service communication with WebClient

Services often need data from each other. For example, the Post Service stores only userId on each post — to show the username on the feed, it must ask the Auth Service. This is done using Spring's WebClient with the @LoadBalanced annotation. When code calls http://authentication-service/api/auth/users/1, Spring intercepts the URL, asks Consul for a healthy instance, and replaces the service name with a real IP:port. All inter-service calls are wrapped in

try/catch with sensible fallbacks — if Auth Service is briefly down, posts still render with user_42 as a placeholder instead of crashing.

Step 5 — Circuit breaker prevents cascading failures

Critical methods are wrapped with Resilience4j's @CircuitBreaker. It monitors the last 10 calls. If 50% or more fail, the circuit **opens** — for the next 60 seconds, all calls skip the broken service and use a fallback method. After 60s the circuit goes **half-open** and allows 3 test calls — if they succeed, the circuit **closes** and normal traffic resumes. Without this, a slow Trending Service could block all post creations and bring down the whole system.

Step 6 — Notifications across services

Notifications are created by the Follow Service but triggered from anywhere. When a user likes a post, the Post Service calls POST http://follow-service/api/notifications via WebClient. The Follow Service saves the notification, and the post owner sees it the next time their frontend polls (every 30 seconds). The three notification types are LIKE, COMMENT (both triggered by Post Service) and FOLLOW (triggered by Follow Service itself).

5. End-to-end flow — when a user likes a post

To see how all the pieces work together, here's what happens when User A likes one of User B's posts:

Step	Where	What happens
1	**Frontend**	User clicks the heart on a post in PostCard. The UI updates immediately (red heart, count + 1) — optimistic update. Then a POST request goes to /api/posts/42/like with the JWT token.
2	**API Gateway**	JwtAuthenticationFilter validates the token, extracts username='john_doe' and userId=1, adds them as X-Username and X-User-Id headers, queries Consul for post-service, and forwards the request.
3	**Post Service**	Reads the headers. Checks if user 1 has already liked post 42 (unique constraint). Saves a new like record. Increments likesCount from 5 to 6 in the posts table.
4	**Post Service → Follow Service**	Sends a notification via WebClient: POST http://follow-service/api/notifications with { senderId: 1, receiverId: 7 (post owner), type: 'LIKE', message: 'john_doe liked your post' }.
5	**Follow Service**	Saves the notification row to the database. Returns 200 OK.
6	**Response back**	Post Service returns the updated post to the gateway, which returns it to the frontend. If anything failed, the UI rolls back to its previous state.
7	**User B's frontend**	Navbar polls /api/notifications/unread-count every 30 seconds. Sees the new count, shows a red badge on the heart icon. User B clicks → sees: 'john_doe liked your post · 2m'.

What this flow demonstrates

- **Single entry point.** Frontend only knows port 8080. It never directly calls any backend service.
- **Stateless auth.** The JWT travels with every request; no server-side session needed.
- **Service discovery.** Gateway asks Consul where post-service lives — no hardcoded IPs.
- **Database per service.** Likes are saved by Post Service in instagram_posts; notifications by Follow Service in instagram_follows. Separate concerns, separate data.
- **Cross-service communication.** Post Service uses WebClient to call Follow Service — wrapped in try/catch so a notification failure doesn't break the like.
- **Eventual consistency.** User B doesn't see the notification instantly — they see it within 30 seconds via polling. That's a deliberate tradeoff vs. running WebSockets.

6. Databases — four separate schemas

Each business service owns its own MySQL database. This means a change to the posts schema only affects the Post Service — no other service has its tables coupled to it. The tradeoff is that joining data across services requires API calls instead of SQL joins.

Database	Tables	Stores
instagram_auth	users	User accounts, BCrypt-hashed passwords, profile data, password-reset tokens.
instagram_posts	posts, post_likes, post_comments, post_hashtags, media_files, video_views	All post data, denormalised like/view counts for fast feed queries, media stored as MEDIUMBLOB up to 16MB.
instagram_follows	follows, notifications	Follower relationships (unique constraint prevents duplicates), notifications with read/unread flag.
instagram_trending	trending_hashtags	Hashtag-to-count mapping. Updated whenever a post containing the hashtag is created.

Design choices worth knowing

• **Denormalised counters.** likes_count and views_count are stored directly on the posts row, so showing the feed doesn't need a COUNT(*) JOIN.

• **Unique constraints.** uk_post_user on post_likes prevents double-liking; uk_follower_following on follows prevents duplicate follow rows.

• **ON DELETE CASCADE.** When a post is deleted, its likes, comments, hashtags, and views are deleted automatically — no orphan rows.

• **Indexes on every foreign key.** All commonly-queried columns (user_id, post_id, created_at, hashtag, receiver_id) have indexes for fast lookups.

• **utf8mb4 charset.** Standard utf8 in MySQL only supports 3-byte characters. utf8mb4 handles emojis correctly in captions and comments.

7. Why microservices — the trade-offs

What we gain

- **Independent deployment.** Update Trending Service without touching anything else.
- **Fault isolation.** If Trending Service crashes, users can still log in, post, like, follow.
- **Independent scaling.** Run 5 copies of Post Service if the feed is hot, while keeping 1 copy of Trending.
- **Technology freedom.** Could swap Trending Service to Redis without changing other services.
- **Clear ownership.** Each domain is in one place — easy to debug, easy for teams to own.

What we give up

- **Cross-service data is harder.** No SQL JOIN across databases. Need API calls + 50ms of latency.
- **More moving parts.** 5 services + gateway + Consul + MySQL = 8 containers to keep healthy.
- **Eventual consistency.** Notifications arrive ~30s after the event, not instantly.
- **Operational overhead.** Need monitoring, distributed logging, circuit breakers, health checks — none of that is required for a monolith.

> **Bottom line:** microservices add complexity in exchange for resilience and independent scaling. For a project this size, a monolith would be simpler. But the whole point of ZKinsta is to demonstrate the patterns — service discovery, gateway routing, circuit breakers, database-per-service, inter-service HTTP — that real production systems at scale rely on.

8. Project at a glance

Metric	Value
Independent microservices	**5**
Separate MySQL databases	**4**
Database tables across the system	**10**
REST API endpoints	**40+**
React components in the frontend	**13**
TypeScript service modules	**6**
Lines of backend Java code	**~3000+**
Lines of frontend TypeScript code	**~4000+**
Lines of CSS across 8 stylesheets	**~2200**
Instagram-style CSS image filters	**12**
Notification types (LIKE, COMMENT, FOLLOW)	**3**
Total containers in Docker Compose	**8**

How to run it

- **One command:** docker-compose up --build — starts everything (MySQL, Consul, 5 services, frontend).
- Wait roughly 60–90 seconds for all containers to be healthy.
- Open http://localhost:3000 for the app.
- Consul UI at http://localhost:8500 — all five services should show green.
- Swagger UI on each service at http://localhost:<port>/swagger-ui.html for testing endpoints directly.

End of summary. This document is a condensed overview — the full technical report with all code, SQL DDL, viva Q&A;, and detailed flow walkthroughs is available as a separate document.